

Информационная система городской телефонной сети

Разбор для защиты: база данных, запросы, CRUD, Rust и SeaORM

Объяснение «с нуля» — что, зачем и где в коде

Содержание

1	Как устроен проект целиком	2
2	База данных: что это и как устроена	2
2.1	Реляционная БД и PostgreSQL простыми словами	2
2.2	Сущности предметной области и их таблицы	2
2.3	Наследование АТС (один из «хитрых» моментов)	3
2.4	Виды ограничений целостности	3
2.5	Триггеры — что это и зачем	4
2.6	Представления (VIEW) и функция расчёта абонплаты	4
3	Запросы к базе	5
3.1	SQL по-простому	5
3.2	Разбор запроса №9: «город-лидер по межгороду»	5
3.3	Разбор запроса №3: «должники»	6
3.4	Параметры и защита от SQL-инъекций	6
3.5	Где живут запросы в коде	6
4	CRUD: создание, чтение, изменение, удаление	6
4.1	Что такое CRUD	6
4.2	Поток одного запроса (например, список абонентов)	6
4.3	Главная идея: один generic-код на все сущности	7
4.4	Права (RBAC)	7
4.5	Как добавить новую сущность (рецепт)	7
5	Rust и SeaORM: как это устроено	7
5.1	Rust в двух словах	7
5.2	Что такое ORM	8
5.3	SeaORM: 4 понятия на примере абонента	8
5.4	Откуда берутся entity: генерация из схемы	8
5.5	SeaORM поверх sqlx: почему и ORM, и сырой SQL	8
5.6	Как find().paginate() превращается в SQL	9
6	Шпаргалка к защите	9

1 Как устроен проект целиком

Проект – это **веб-приложение** для городской телефонной сети (ГТС). Оно состоит из трёх частей («слоёв»), которые общаются друг с другом:

Слой	Что делает	Технологии
База данных	Хранит все данные: АТС, абонентов, номера, счета, звонки. Сама следит за правильностью данных.	PostgreSQL
Backend (сервер)	Принимает запросы из браузера по HTTP, проверяет права, ходит в БД, отдаёт JSON.	Rust, Axum, SeaORM, sqlx
Frontend (сайт)	То, что видит пользователь в браузере: таблицы, формы, кнопки.	Vue 3, TypeScript

Поток данных всегда один и тот же: **браузер** → (HTTP-запрос) → **backend** → (SQL-запрос) → **база данных** → и обратно ответом.

Где смотреть. Карта репозитория:

- backend/migrations/ – SQL-файлы, которые создают базу (таблицы, триггеры, представления).
- backend/seeds/dev_seed.sql – демо-данные.
- backend/entity/ – «отражение» таблиц в виде структур Rust (SeaORM).
- backend/server/src/ – код сервера (логика, запросы, права).
- frontend/src/ – сайт на Vue.
- docs/schema.dbml – визуальная схема БД (открыть на dbdiagram.io).

2 База данных: что это и как устроена

2.1 Реляционная БД и PostgreSQL простыми словами

Реляционная база данных – это набор **таблиц**. Каждая таблица – как лист Excel: есть колонки (поля) и строки (записи). Например, таблица `subscriber` (абонент) – это список людей, где у каждого есть фамилия, имя, дата рождения и т.д.

У каждой строки есть **первичный ключ** (PRIMARY KEY, обычно колонка `id`) – уникальный номер строки, по которому на неё можно сослаться. Таблицы связаны **внешними ключами** (FOREIGN KEY): например, у абонента есть поле `phone_number_id`, которое указывает на строку в таблице `phone_number`. Так данные «склеиваются» без дублирования.

PostgreSQL – это конкретная СУБД (программа, которая управляет такой базой). Она умеет не только хранить данные, но и сама проверять правила (ограничения и триггеры), о чём ниже.

2.2 Сущности предметной области и их таблицы

Предметная область («что моделируем») – городская телефонная сеть. Главные понятия превратились в таблицы:

Таблица	Что хранит
<code>pbx</code>	АТС (телефонная станция). Бывает трёх типов: городская, ведомственная, учрежденческая.
<code>pbx_city</code> / <code>pbx_department</code> / <code>pbx_institution</code>	Доп. поля для каждого типа АТС (см. «наследование» ниже).

Таблица	Что хранит
phone_number	Телефонный номер: на какой АТС, тип линии (основной/параллельный/спаренный), есть ли межгород.
subscriber	Абонент (человек): ФИО, пол, дата рождения, льготник или нет. Ссылается на номер.
address	Адрес: индекс, район, улица, дом, квартира.
call_record	Журнал звонков (CDR): откуда, тип, город назначения, длительность, стоимость.
tariff, invoice, payment, penalty, notification, billing_settings	Биллинг: тарифы, счета, оплаты, пени, уведомления, настройки.
installation_queue	Очередь на установку телефона (обычная/льготная).
public_phone	Таксофоны и общественные телефоны.
customer	Аккаунт абонента для личного кабинета (логин/пароль).
app_user, role, permission, ...	Сотрудники (операторы) и их права (ролевая система).

Где смотреть. Все таблицы создаются в backend/migrations/0001...0014_*.sql. Каждый файл — отдельный «шаг» создания базы. Визуально всю схему со связями видно в docs/schema.dbml.

2.3 Наследование АТС (один из «хитрых» моментов)

По заданию АТС бывает 3 типов, и у каждого типа — свои атрибуты. Это классическая ситуация «наследования». Мы решили её приёмом **class-table inheritance**:

- общая таблица pbx хранит то, что есть у всех АТС (имя, код, район, ёмкость, каналы);
- три таблицы-«наследника» (pbx_city, pbx_department, pbx_institution) связаны с pbx один-к-одному (их ключ pbx_id = pbx.id) и хранят только специфичные поля (например, у городской — region_code, у ведомственной — department_name).

Чтобы нельзя было приписать городской АТС «ведомственные» поля, стоит **триггер** (см. ниже), который сверяет тип.

2.4 Виды ограничений целостности

«Целостность» = данные всегда корректны. По заданию это требование №1, и почти всё вынесено на уровень БД (а не в код), чтобы никакой кривой запрос не испортил данные. Виды ограничений:

Ограничение	Что гарантирует (пример из проекта)
PRIMARY KEY	Уникальный id строки.
FOREIGN KEY	Ссылка ведёт на существующую строку (нельзя создать абонента с несуществующим номером).

Ограничение	Что гарантирует (пример из проекта)
UNIQUE	Нет дублей (номер телефона уникален; логин пользователя уникален).
NOT NULL	Поле обязательно (у абонента обязана быть фамилия).
CHECK	Условие на значения. Напр. chk_privilege: если абонент льготник — у него указан вид льготы, если простой — не указан.
ENUM (перечисление)	Поле принимает только значения из списка (тип линии = только main/parallel/paired).
Частичный UNIQUE	Хитрость: только один город помечен «свой» (is_home).

2.5 Триггеры — что это и зачем

Триггер — это маленькая функция, которая **автоматически срабатывает** при изменении таблицы (вставка/обновление/удаление). Нужна там, где обычного CHECK мало — когда правило затрагивает несколько строк или таблиц.

В проекте 7 триггеров (файл 0008_triggers.sql). Например, правило «на основном номере максимум 1 абонент, на спаренном — 2, на параллельном — много» нельзя выразить простым CHECK (надо посчитать другие строки). Делаем триггером:

```
CREATE OR REPLACE FUNCTION trg_subscriber_count_check() RETURNS trigger AS $$
DECLARE lt line_type; cnt INTEGER; max_subs INTEGER;
BEGIN
    SELECT line_type INTO lt FROM phone_number WHERE id = NEW.phone_number_id;
    SELECT count(*) INTO cnt FROM subscriber
        WHERE phone_number_id = NEW.phone_number_id AND id <> NEW.id;
    max_subs := CASE lt WHEN 'main' THEN 1 WHEN 'paired' THEN 2 WHEN 'parallel' THEN
2147483647 END;
    IF cnt + 1 > max_subs THEN
        RAISE EXCEPTION 'Number % (line type %) allows at most % subscriber(s)', ...;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER subscriber_count_check BEFORE INSERT OR UPDATE ON subscriber
FOR EACH ROW EXECUTE FUNCTION trg_subscriber_count_check();
```

Читается так: «**перед** (BEFORE) вставкой/обновлением строки в subscriber посчитай, сколько абонентов уже на этом номере; если станет больше лимита для типа линии — брось ошибку (RAISE EXCEPTION)». NEW — это новая строка, которую пытаются записать.

Наши 7 триггеров (что проверяют):

1. соответствие подтипа АТС её типу (городская ≠ ведомственные поля);
2. межгород только у городских АТС; у замкнутых сетей — none;
3. лимит абонентов на номер по типу линии (пример выше);
4. параллельные/спаренные абоненты — обязательно в одном доме;
5. автоматическая синхронизация статуса номера (free/active) при появлении/удалении абонента.

Важно. На защите частый вопрос: «почему не проверять в коде?» Ответ: правило в БД работает **всегда**, даже если кто-то полезет в базу напрямую или другой программой. Это надёжнее и соответствует требованию «целостность максимально на уровне БД».

2.6 Представления (VIEW) и функция расчёта абонплаты

Представление (VIEW) — это «сохранённый запрос», к которому можно обращаться как к таблице. Удобно, когда один и тот же сложный SELECT нужен в разных местах.

Главный пример — `v_subscriber_debt` (долг абонента). Он собирает долг из трёх источников (неоплаченные счета за абонплату, за межгород, и пени) в одну «виртуальную таблицу»:

```
CREATE VIEW v_subscriber_debt AS
SELECT s.id AS subscriber_id,
       COALESCE(sub.amt,0) AS subscription_debt,
       COALESCE(inter.amt,0) AS intercity_debt,
       COALESCE(pen.amt,0) AS penalty_debt,
       COALESCE(sub.amt,0)+COALESCE(inter.amt,0)+COALESCE(pen.amt,0) AS total_debt,
       LEAST(sub.oldest, inter.oldest) AS oldest_due_date
FROM subscriber s
LEFT JOIN (SELECT subscriber_id, sum(amount) amt, min(due_date) oldest
           FROM invoice WHERE kind='subscription' AND status IN ('pending','overdue')
           GROUP BY subscriber_id) sub ON sub.subscriber_id = s.id
LEFT JOIN (...intercity...) inter ON ...
LEFT JOIN (SELECT subscriber_id, sum(amount) amt FROM penalty WHERE NOT paid
           GROUP BY subscriber_id) pen ON ...;
```

Теперь в любом запросе про должников мы просто пишем `FROM v_subscriber_debt` — и не повторяем эту логику.

Ещё есть функция `fn_subscriber_monthly_fee(id)` — считает абонплату по тарифу (тип линии × наличие межгорода) и применяет скидку 50% льготникам.

Где смотреть. Представления и функция — в `0009_views.sql`. Триггеры — в `0008_triggers.sql`.
Краткое описание модели — в `docs/schema.md`.

3 Запросы к базе

3.1 SQL по-простому

Запрос на чтение данных пишется командой `SELECT`. Основные кусочки:

Часть	Смысл
<code>SELECT a, b</code>	какие колонки вернуть
<code>FROM t</code>	из какой таблицы
<code>JOIN t2 ON ...</code>	приклеить связанную таблицу (напр. к номеру — его АТС)
<code>WHERE ...</code>	условие-фильтр (только нужные строки)
<code>GROUP BY x</code>	сгруппировать строки (напр. по городу) для подсчёта
<code>count(*), sum(...)</code>	агрегаты: посчитать количество / сумму внутри группы
<code>ORDER BY ... DESC</code>	сортировка

3.2 Разбор запроса №9: «город-лидер по межгороду»

Один из 13 запросов варианта. Задача: найти город, в который больше всего междугородних звонков.

```
SELECT c.name AS city, count(*) AS calls
FROM call_record cr
JOIN city c ON c.id = cr.dest_city_id
WHERE cr.call_type = 'intercity'
GROUP BY c.name
ORDER BY calls DESC;
```

Построчно: берём журнал звонков `call_record`, приклеиваем к каждому звонку его город (`JOIN city`), оставляем только межгород (`WHERE`), группируем по названию города (`GROUP BY`), в каждой группе считаем число звонков (`count(*)`), сортируем по убыванию. Первая строка — город-лидер.

3.3 Разбор запроса №3: «должники»

Более «боевой» запрос — с параметрами и представлением `v_subscriber_debt`:

```
SELECT vf.last_name, vf.number, vf.pbx_name,
       d.subscription_debt, d.intercity_debt, d.total_debt,
       (CURRENT_DATE - d.oldest_due_date) AS days_overdue
FROM v_subscriber_debt d
JOIN v_subscriber_full vf ON vf.id = d.subscriber_id
WHERE d.total_debt > 0
      AND ($1::bigint IS NULL OR vf.pbx_id = $1)           -- фильтр по АТС
      AND ($3::int IS NULL OR (CURRENT_DATE - d.oldest_due_date) >= $3) -- просрочка > N
      AND ($5::numeric IS NULL OR d.total_debt >= $5)      -- долг >= суммы
ORDER BY d.total_debt DESC;
```

Хитрость с `($1 IS NULL OR ...)`: если параметр не передан (NULL) — условие «выключается» и фильтр не применяется. Так один запрос обслуживает все варианты («по АТС / по всей сети / по сроку / по размеру долга») без копирования кода.

3.4 Параметры и защита от SQL-инъекций

`$1`, `$2`, ... — это **параметры** (placeholders). Значения подставляет драйвер **отдельно** от текста запроса (**prepared statements**). Это защищает от **SQL-инъекций**: даже если пользователь введёт `' ; DROP TABLE ...`, это попадёт как обычная строка-значение, а не как команда. Мы **никогда** не склеиваем SQL из строк руками.

3.5 Где живут запросы в коде

13 аналитических запросов — в `backend/server/src/analytics.rs`. Каждый — это маленький обработчик: берёт параметры из URL, строит SQL, оборачивает результат в JSON прямо средствами PostgreSQL (`json_agg`), отдаёт массив.

Где смотреть.

- Аналитика (Q1–Q13): `backend/server/src/analytics.rs`
- Соответствие запрос → таблицы: таблица в конце `docs/schema.md`
- Произвольные запросы пользователя: `backend/server/src/raw_query.rs` — разрешён только один SELECT, выполняется в режиме «только чтение» (READ ONLY) с тайм-аутом, чтобы пользователь ничего не сломал.

4 CRUD: создание, чтение, изменение, удаление

4.1 Что такое CRUD

CRUD = Create / Read / Update / Delete — четыре базовые операции над любой сущностью. Для каждой таблицы (АТС, абоненты, тарифы...) нужны: список (с постранично — пагинацией), просмотр одной записи, создание, изменение, удаление. По заданию это требование №4, и важна мысль «переиспользование кода» — не писать одно и то же 16 раз.

4.2 Поток одного запроса (например, список абонентов)

1. Браузер запрашивает `GET /api/subscribers?page=1&page_size=20`.
2. Сервер (Ахум) находит обработчик, проверяет **право** `subscriber:read`.
3. Обработчик через SeaORM делает `Entity::find().paginate(...)` — SeaORM превращает это в SQL `SELECT ... LIMIT 20 OFFSET 0`.
4. PostgreSQL возвращает строки; SeaORM превращает их в структуры Rust; сервер отдаёт JSON.
5. Vue рисует таблицу.

4.3 Главная идея: один generic-код на все сущности

Вместо 16 копий написан **один** обобщённый («generic») набор обработчиков. Он работает с **любой** сущностью E. Сердце — функция `crud_routes::<E>()`, которая выдаёт 5 маршрутов:

```
pub fn crud_routes<E>() -> Router<AppState>
where E: Resource, /* ...ограничения на тип... */ {
    Router::new()
        .route("/", get(list::<E>).post(create::<E>))
        .route("/{id}", get(get_one::<E>).put(update::<E>).delete(delete::<E>))
}
```

Resource — наш маленький «интерфейс», который связывает сущность с её именем-правом:

```
pub trait Resource: EntityTrait { const NAME: &'static str; }
```

А подключаются все сущности в одном месте — списком (`resources.rs`):

```
.nest("/subscribers", crud_routes::<entity::subscriber::Entity>())
.nest("/pbx", crud_routes::<entity::pbx::Entity>())
.nest("/cities", crud_routes::<entity::city::Entity>())
// ...и так все 16 ресурсов
```

Внутри `list` есть пагинация и проверка права:

```
user.require(&format!("{}", E::NAME)); // право subscriber:read и т.п.
let paginator = E::find().paginate(&st.db, page_size);
let items = paginator.fetch_page(page - 1).await?; // SELECT ... LIMIT .. OFFSET ..
```

Пагинация — это выдача данных страницами (по 20 строк), чтобы не тянуть всю таблицу разом. SeaORM делает это сам через LIMIT/OFFSET.

4.4 Права (RBAC)

Каждая операция требует право вида сущность:действие (`subscriber:read`, `pbx:create`, ...). Права хранятся в БД и собираются в **роли** роли назначаются пользователям. Суперадмин может всё настроить через админку (это требование №7 — роли «не прибиты гвоздями»). Проверка — строка `user.require("subscriber:create")`? внутри обработчика.

Где смотреть.

- Generic CRUD: `backend/server/src/crud.rs`
- Подключение сущностей: `backend/server/src/resources.rs`
- Права/роли (управление): `backend/server/src/admin.rs`, таблицы `role/permission`
- На фронте: один компонент `frontend/src/views/CrudView.vue` рисует таблицу+форму для любой сущности по «описанию» из `frontend/src/config/resources.ts`.

4.5 Как добавить новую сущность (рецепт)

1. Добавить таблицу в новую миграцию.
2. Сгенерировать entity (см. след. раздел) — появится `entity/src/новая.rs`.
3. В `resources.rs` дописать `resource!(новая, "новая")` и `.nest("/новая", crud_routes::<...>())`.
4. Добавить права `новая:read/create/...` (миграция) — и всё, CRUD готов и на бэке, и на фронте.

5 Rust и SeaORM: как это устроено

5.1 Rust в двух словах

Rust — компилируемый язык с очень строгим **компилятором**, который ловит большинство ошибок **до запуска** (несовпадение типов, обращение к пустому значению, гонки данных). Если проект скомпилировался — он, как правило, уже не «упадёт» на ерунде. Главные идеи:

- **Сильная типизация**: у каждого значения известен тип; нельзя случайно сложить число и строку.

- **Владение и заимствование** (ownership/borrowing): компилятор сам следит за памятью без «сборщика мусора», поэтому Rust быстрый и безопасный.
- `Result<T, E>` вместо исключений: функция явно возвращает «успех или ошибку», и её приходится обработать (отсюда ? в коде — «если ошибка, верни её выше»).

5.2 Что такое ORM

ORM (Object-Relational Mapping) — «переходник» между таблицами БД и объектами/структурами языка. Без ORM вы пишете SQL руками и сами раскладываете результат по полям. С ORM таблица превращается в структуру, а строки — в её экземпляры; типичные операции (найти по id, вставить, обновить) пишутся на языке, а ORM сам генерирует SQL. Преподаватель рекомендовал ORM (как Hibernate в Java) — у нас это **SeaORM**.

5.3 SeaORM: 4 понятия на примере абонента

Для каждой таблицы SeaORM описывает её четырьмя связанными вещами (файл `entity/src/subscriber.rs`):

Понятие	Что это
Model	структура Rust = одна строка таблицы (поля <code>id</code> , <code>last_name</code> , <code>gender</code> , ...). Именно её мы отдаём в JSON.
Entity	сама таблица (к ней зовём <code>find()</code> , <code>find_by_id()</code> , <code>delete_by_id()</code>).
ActiveModel	«редактируемая» строка для вставки/обновления: у каждого поля состояние «задано / не задано».
Column	перечисление колонок (для сортировки/фильтров), <code>Relation</code> — связи с другими таблицами.

Сам Model выглядит так (сокращённо):

```
#[derive(Clone, Debug, DeriveEntityModel, Serialize, Deserialize)]
#[sea_orm(table_name = "subscriber")]
pub struct Model {
    #[sea_orm(primary_key)] pub id: i64,
    pub last_name: String,
    pub gender: Gender,           // это наш ENUM-тип
    pub birth_date: Date,
    pub phone_number_id: i64,     // внешний ключ на phone_number
    // ...
}
```

`#[derive(...)]` и `#[sea_orm(...)]` — это **макросы/атрибуты**: они говорят SeaORM «сгенерируй за меня код для работы с этой таблицей». Поэтому одной структуры достаточно, чтобы появились Entity, ActiveModel и т.д.

5.4 Откуда берутся entity: генерация из схемы

Мы не пишем эти структуры руками. Сначала создаём таблицы SQL-миграциями, потом **генерируем** entity прямо из живой базы командой:

```
sea-orm-cli generate entity -u postgres://... -o entity/src --lib --with-serde both
```

После генерации запускаем небольшой скрипт `backend/scripts/postprocess_entities.py`, который дорабатывает сгенерированный код: делает так, чтобы JSON отдавал значения ENUM в «нижнем регистре как в БД» (`male`, а не `Male`) и чтобы при создании можно было прислать частичные данные. Это инженерная мелочь, но на защите полезно знать, зачем скрипт нужен.

5.5 SeaORM поверх sqlx: почему и ORM, и сырой SQL

SeaORM построен поверх библиотеки **sqlx** (она держит **пул соединений** с базой и реально отправляет запросы). Мы используем оба инструмента осознанно:

- **SeaORM** — для типового **CRUD** (мало кода, безопасно, переиспользуемо).
- **sqlx** (сырой SQL) — для 13 аналитических запросов, личного кабинета и произвольных запросов, где сложный SQL писать напрямую проще и нагляднее, чем через «конструктор» ORM.

Достаём пул sqlx прямо из SeaORM-соединения:

```
pub fn pool(&self) -> &sqlx::PgPool { self.db.get_postgres_connection_pool() }
```

5.6 Как find().paginate() превращается в SQL

Когда мы пишем на Rust:

```
let paginator = entity::subscriber::Entity::find().paginate(&db, 20);
let page1 = paginator.fetch_page(0).await?; // страница 0 = первые 20
```

SeaORM строит и выполняет примерно такой SQL:

```
SELECT "subscriber"."id", "subscriber"."last_name", ...
FROM "subscriber"
ORDER BY "subscriber"."id" ASC
LIMIT 20 OFFSET 0;
```

То есть `.find()` → `SELECT FROM subscriber, .paginate(..., 20).fetch_page(0)` → `LIMIT 20 OFFSET 0`. Результат (строки) SeaORM раскладывает обратно в `Vec<Model>`, который сервер сериализует в JSON. `.await?` — потому что обращение к базе **асинхронное** (сервер не «зависает», ожидая ответ БД, а обслуживает другие запросы).

Где смотреть.

- Entity (сгенерированы): `backend/entity/src/*.rs`
- Подключение к БД и пул: `backend/server/src/db.rs, state.rs`
- Generic CRUD на SeaORM: `backend/server/src/crud.rs`
- Зависимости и версии: `backend/server/Cargo.toml`

6 Шпаргалка к защите

Вопрос	Короткий ответ + где код
Где обеспечивается целостность данных?	На уровне БД: PK/FK/UNIQUE/CHECK/ENUM + 7 триггеров. <code>migrations/0008_triggers.sql</code> .
Зачем триггеры, а не проверки в коде?	Работают всегда, при любом доступе к БД; требование «целостность максимально в БД».
Как реализованы 13 запросов варианта?	Параметризованный SQL в <code>analytics.rs</code> ; сложную логику долга вынесли в VIEW <code>v_subscriber_debt</code> .
Как защищены от SQL-инъекций?	Только prepared statements (\$1, \$2); строки руками не склеиваем.
Что такое CRUD и где переиспользование?	Один <code>generic crud_routes::<E>()</code> на все 16 сущностей. <code>crud.rs</code> + <code>resources.rs</code> .
Что такое пагинация?	Выдача страницами через LIMIT/OFFSET; в SeaORM — <code>.paginate().fetch_page()</code> .
Как устроена авторизация/роли?	Права сущность:действие в БД, роли настраивает суперадмин. <code>admin.rs</code> , таблицы <code>role/permission</code> .
Что такое SeaORM?	ORM для Rust: таблица → Model/Entity/ActiveModel. Генерируем из схемы <code>sea-orm-cli</code> .

Вопрос	Короткий ответ + где код
ORM или сырой SQL?	Оба: SeaORM для CRUD, sqlx для аналитики и кабинета. SeaORM работает поверх sqlx.
Конфигурация подключения к БД?	В <code>backend/config.toml</code> (host/port/user/password/name) + переопределение через переменные окружения.

Полная визуальная схема БД — `docs/schema.dbml` (импортировать на dbdiagram.io). Описание API — `docs/api.md`.